

展開相機體驗之旅

來源：<https://codelabs.developers.google.com/codelabs/android-camera-foldables?hl=zh-tw>

步驟數：7

這些年來，Android 裝置不斷演進，除了推出了各種尺寸的機種，外觀、螢幕和功能也更加多元。不過，從一開始，使用手機拍照就一直是最重要的用途之一。時至今日，相機功能仍是消費者購買手機時，考慮的其中一項主因。在本程式碼研究室中，您將瞭解如何善用摺疊式裝置帶來的各種用途，為使用者提供絕佳的相機體驗！

目錄

1. [1. 事前準備](#)
2. [2. 做好準備](#)
3. [3. 執行並觀察](#)
4. [4. 瞭解 Jetpack WindowManager](#)
5. [5. 實作後置自拍模式](#)
6. [6. 實作免手持模式](#)
7. [7. 恭喜！](#)

步驟 1 · 約 2 分鐘

1. 事前準備

摺疊式裝置有何特別之處？

摺疊式裝置是本世代難得的創舉，不僅能提供獨特的使用體驗，還能帶來獨有的機會，讓您透過可免持使用的桌面 UI 這類差異化功能，滿足使用者的各種需求。

必要條件

- 具備開發 Android 應用程式的基本知識
- 具備 Hilt 依附元件插入架構的基本知識

建構項目

在本程式碼研究室中，您會建構一款相機應用程式，並針對摺疊式裝置採用最佳化的版面配置。



我們會從基本相機應用程式開始，這類應用程式不會回應任何裝置型態，也不會利用效能更佳的后置鏡頭提升自拍效果。之後，您需更新原始碼，讓預覽畫面在裝置展開時移至較小的螢幕，並對設為桌面模式的手機做出回應。

雖然這個 API 最適合的用途就是製作相機應用程式，但本程式碼研究室介紹的這兩項功能可應用於任何應用程式。

課程內容

- 如何使用 Jetpack Window Manager 對裝置型態變化做出回應
- 如何將應用程式移至摺疊式裝置的小螢幕

軟硬體需求

- 最新版 Android Studio
- 摺疊式裝置或摺疊式裝置模擬器

注意：開發摺疊式裝置的相機應用程式時，有幾件事需要列入考量，像是如何支援可調整大小的介面，以及處理螢幕方向和顯示比例。如要瞭解詳情，請前往 <https://developer.android.com/training/camera2/camera-preview>。

在本程式碼研究室中，您將使用 Camera ViewFinder，這是一個小型程式庫，與現有的 Camera2 程式碼集完全相容，可為您處理大部分的細微差異。詳情請造訪

<https://developer.android.com/reference/androidx/camera/viewfinder/package-summary>。

步驟 2 · 約 5 分鐘

2. 做好準備

取得範例程式碼

1. 如果您已安裝 Git，只要執行下列指令即可。如要檢查 Git 是否已安裝完成，請在終端機或指令列中輸入 `git --version`，並確認該指令可正確執行。

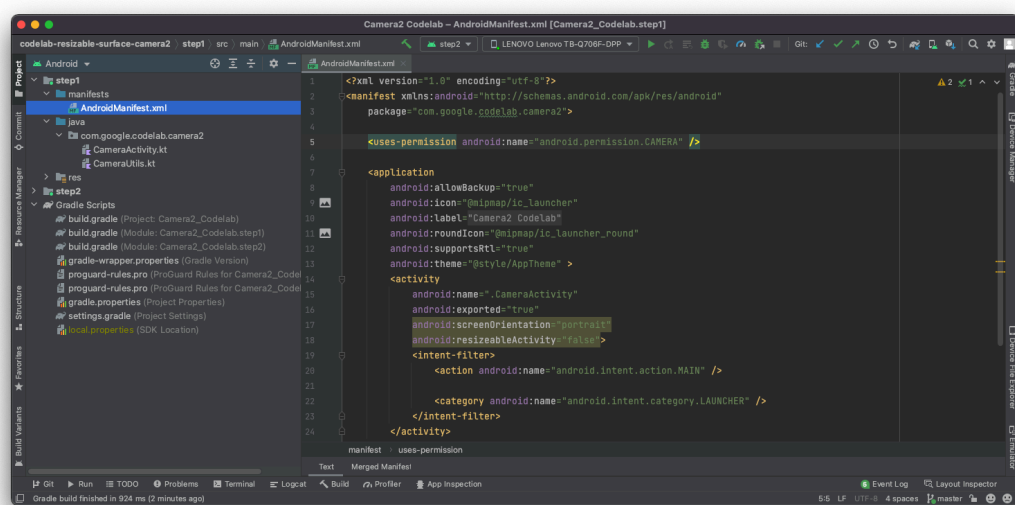
```
git clone https://github.com/android/large-screen-codelabs.git
```

2. 選用：如果您沒有 Git，可以點選下方按鈕，下載這個程式碼研究室的所有程式碼：

開啟第一個模組

- 在 Android Studio 中，開啟 `/step1` 下的第一個模組。

注意：如有任何問題，建議您按照「[更新 IDE 和 SDK 工具](#)」中的建議操作。



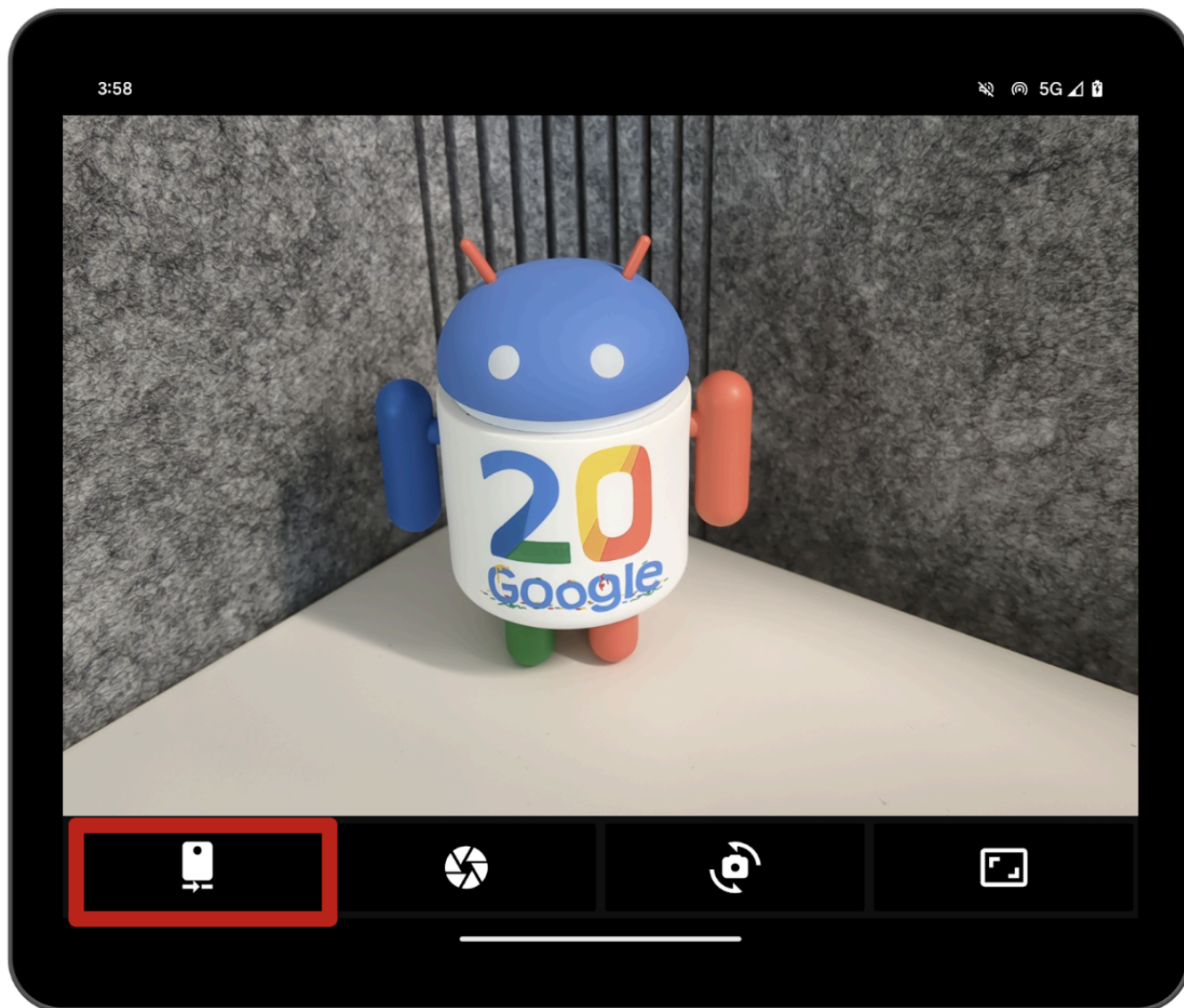
如果系統要求您使用最新版 Gradle，請直接更新。

步驟 3 · 約 2 分鐘

3. 執行並觀察

1. 執行模組 `step1` 中的程式碼。

如您所見，這是一款簡單的相機應用程式。您可以在前置與後置鏡頭間切換，也可以調整顯示比例。不過，左側的第一個按鈕目前沒有任何作用，但這會是「後置鏡頭自拍」模式的進入點。



2. 現在，請嘗試將裝置調到半開的位置，亦即轉軸並非完全平展或閉合，而是呈 90 度角。

如您所見，應用程式不會對不同裝置型態做出回應，因此版面配置不會改變，轉軸仍位於觀景窗中央。

4. 瞭解 Jetpack WindowManager

Jetpack WindowManager 程式庫可協助應用程式開發人員打造最佳的摺疊式裝置體驗。其中包含 **FoldingFeature** 類別，用於說明彈性顯示畫面中的摺疊方式，或是兩個實體顯示面板之間的轉軸。其 API 可讓您存取與裝置相關的重要資訊：

- 如果轉軸呈 180 度打開，**state()** 會傳回 **FLAT**；否則會傳回 **HALF_OPENED**。
- 如果 **FoldingFeature** 寬度大於高度，**orientation()** 會傳回 **FoldingFeature.Orientation.HORIZONTAL**；否則會傳回 **FoldingFeature.Orientation.VERTICAL**。
- **bounds()** 用於提供 **FoldingFeature** 的邊界 (採 **Rect** 格式)。

FoldingFeature 類別包含 **occlusionType()** 或 **isSeparating()** 等其他資訊，但本程式碼研究室在這方面不會深入探討。

從 **1.2.0-beta01** 版開始，程式庫會使用 **WindowAreaController**。這個 API 可讓「後置顯示模式」將目前視窗移至與後置鏡頭對齊的螢幕，非常適合以後置鏡頭自拍等其他用途！

新增依附元件

- 如要在應用程式中使用 Jetpack WindowManager，您需在模組層級的 **build.gradle** 檔案中加入下列依附元件：

step1/build.gradle

```
def work_version = '1.2.0-beta01'
implementation "androidx.window:window:$work_version"
implementation "androidx.window:window-java:$work_version"
implementation "androidx.window:window-core:$work_version"
```

您現在可以在應用程式中存取 **FoldingFeature** 和 **WindowAreaController**。只要使用這兩個類別，就能打造極致的摺疊式裝置相機體驗！

5. 實作後置自拍模式

首先從後置顯示模式著手。

支援此模式的 API 是 `WindowAreaController`，其可在裝置螢幕和顯示區域間移動視窗，並提供相關資訊。

您也可以透過這個 API 查詢清單，瞭解目前可與哪些 `WindowAreaInfo` 互動。

利用 `WindowAreaInfo`，您可以存取 `WindowAreaSession`，這個介面代表使用中的視窗區域功能，以及特定 `WindowAreaCapability` 的可用性狀態。

1. 請在 `MainActivity` 中宣告這些變數：

step1/MainActivity.kt

```
private lateinit var windowAreaController: WindowAreaController
private lateinit var displayExecutor: Executor
private var rearDisplaySession: WindowAreaSession? = null
private var rearDisplayWindowAreaInfo: WindowAreaInfo? = null
private var rearDisplayStatus: WindowAreaCapability.Status =
    WindowAreaCapability.Status.WINDOW_AREA_STATUS_UNSUPPORTED
private val rearDisplayOperation =
    WindowAreaCapability.Operation.OPERATION_TRANSFER_ACTIVITY_TO_AREA
```

2. 接著在 `onCreate()` 方法中初始化這些變數：

step1/MainActivity.kt

```
displayExecutor = ContextCompat.getMainExecutor(this)
windowAreaController = WindowAreaController.getOrCreate()

lifecycleScope.launch(Dispatchers.Main) {
    lifecycle.repeatOnLifecycle(Lifecycle.State.STARTED) {
        windowAreaController.windowAreaInfos
            .map{info->info.firstOrNull{it.type==WindowAreaInfo.Type.TYPE_REAR_FACING}}
            .onEach { info -> rearDisplayWindowAreaInfo = info }
            .map{it?.getCapability(rearDisplayOperation)?.status?:
                WindowAreaCapability.Status.WINDOW_AREA_STATUS_UNSUPPORTED }
            .distinctUntilChanged()
            .collect {
                rearDisplayStatus = it
                updateUI()
            }
    }
}
```

3. 現在實作 `updateUI()` 函式，根據目前狀態啟用/停用後置自拍按鈕：

step1/MainActivity.kt


```

private fun updateUI() {
    if(rearDisplaySession != null) {
        binding.rearDisplay.isEnabled = true
        // A session is already active, clicking on the button will disable it
    } else {
        when(rearDisplayStatus) {
            WindowAreaCapability.Status.WINDOW_AREA_STATUS_UNSUPPORTED -> {
                binding.rearDisplay.isEnabled = false
                // RearDisplay Mode is not supported on this device"
            }
            WindowAreaCapability.Status.WINDOW_AREA_STATUS_UNAVAILABLE -> {
                binding.rearDisplay.isEnabled = false
                // RearDisplay Mode is not currently available
            }
            WindowAreaCapability.Status.WINDOW_AREA_STATUS_AVAILABLE -> {
                binding.rearDisplay.isEnabled = true
                // You can enable RearDisplay Mode
            }
            WindowAreaCapability.Status.WINDOW_AREA_STATUS_ACTIVE -> {
                binding.rearDisplay.isEnabled = true
                // You can disable RearDisplay Mode
            }
            else -> {
                binding.rearDisplay.isEnabled = false
                // RearDisplay status is unknown
            }
        }
    }
}
}

```

最後這步是選用步驟，但可協助您瞭解 `WindowAreaCapability` 的所有可能狀態。

- 現在實作 `toggleRearDisplayMode` 函式，這個函式會關閉工作階段 (如果相關功能已啟用的話)，或呼叫 `transferActivityToWindowArea` 函式：

step1/CameraViewModel.kt

```
private fun toggleRearDisplayMode() {
    if(rearDisplayStatus == WindowAreaCapability.Status.WINDOW_AREA_STATUS_ACTIVE) {
        if(rearDisplaySession == null) {
            rearDisplaySession =
rearDisplayWindowAreaInfo?.getActiveSession(rearDisplayOperation)
        }
        rearDisplaySession?.close()
    } else {
        rearDisplayWindowAreaInfo?.token?.let { token ->
            windowAreaController.transferActivityToWindowArea(
                token = token,
                activity = this,
                executor = displayExecutor,
                windowAreaSessionCallback = this
            )
        }
    }
}
```

請注意，`MainActivity` 會做為 `WindowAreaSessionCallback` 使用。

Rear Display API 支援使用事件監聽器的方法：當您要求將內容移至其他螢幕時，會啟動透過事件監聽器的 `onSessionStarted()` 方法傳回的工作階段。如果您想改為返回內部 (及較大) 的螢幕，請關閉工作階段，而後您會在 `onSessionEnded()` 方法中收到確認訊息。如要建立這類事件監聽器，您需實作 `WindowAreaSessionCallback` 介面。

5. 修改 `MainActivity` 宣告，使其實作 `WindowAreaSessionCallback` 介面：

step1/MainActivity.kt

```
class MainActivity : AppCompatActivity(), WindowAreaSessionCallback
```

現在，請在 `MainActivity` 中實作 `onSessionStarted` 和 `onSessionEnded` 方法。這些回呼方法非常實用，可讓您輕鬆接收工作階段狀態的通知，並據此更新應用程式。

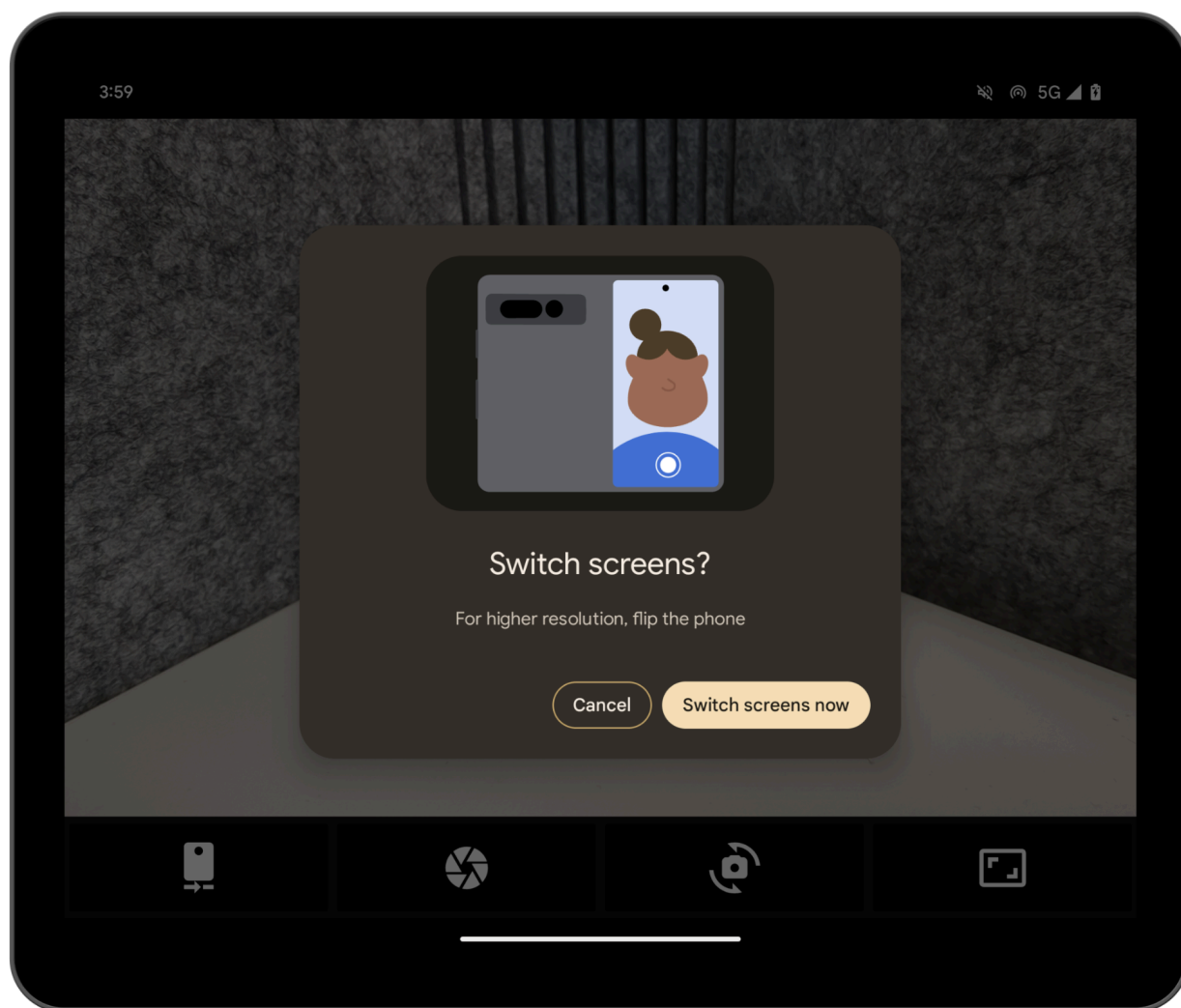
但為了簡單起見，這次我們只需檢查函式主體是否有任何錯誤，並將狀態記錄下來即可。

step1/MainActivity.kt

```
override fun onSessionEnded(t: Throwable?) {
    if(t != null) {
        Log.d("Something was broken: ${t.message}")
    }else{
        Log.d("rear session ended")
    }
}

override fun onSessionStarted(session: WindowAreaSession) {
    Log.d("rear session started [session=$session]")
}
```

6. 建構並執行應用程式。如果您之後展開裝置並輕觸後置顯示按鈕，系統會出現如下的提示訊息：



7. 選取「Switch screens now」，即可看到內容已移至外螢幕！

6. 實作免手持模式

現在可以來設計適合摺疊式裝置使用的應用程式了：也就是根據摺疊方向，將內容移到裝置的一側或轉軸上方。為此，您需在 `FoldingStateActor` 中執行操作，讓程式碼與 `Activity` 分離，更為清楚易懂。

注意：在這個環節中，Hilt 已完成實作，因此您可直接在建構函式中取得所有依附元件。如要進一步瞭解 Hilt 和依附元件在 Android 的插入作業，請參閱「[在 Android 應用程式中使用 Hilt](#)」程式碼研究室。

這個 API 的核心部分在於 `WindowInfoTracker` 介面，此介面是透過需要 `Activity` 的靜態方法建立而成：

step1/CameraCodelabDependencies.kt

```
@Provides
fun provideWindowInfoTracker(activity: Activity) =
    WindowInfoTracker.getOrCreate(activity)
```

您不需要編寫上述程式碼，因為這個程式碼已經存在，但瞭解 `WindowInfoTracker` 的建構方式會很有幫助。

1. 如要監聽任何視窗異動，請在 `Activity` 的 `onResume()` 方法中監聽：

step1/MainActivity.kt

```
lifecycleScope.launch {
    foldingStateActor.checkFoldingState(
        this@MainActivity,
        binding.viewFinder
    )
}
```

2. 現在開啟 `FoldingStateActor` 檔案，是時候填入 `checkFoldingState()` 方法了。

如您所見，此方法會在 `Activity` 的 `RESUMED` 階段執行，並且利用 `WindowInfoTracker` 監聽版面配置的變化。

step1/FoldingStateActor.kt

```
windowInfoTracker.windowLayoutInfo(activity)
    .collect { newLayoutInfo ->
        activeWindowLayoutInfo = newLayoutInfo
        updateLayoutByFoldingState(cameraViewfinder)
    }
```

藉由使用 `WindowInfoTracker` 介面，您可以呼叫 `windowLayoutInfo()` 來收集 `WindowLayoutInfo` 的 `Flow`，其中包含 `DisplayFeature` 內的所有可用資訊。

最後一步是回應這些更動，並據此移動內容。請在 `updateLayoutByFoldingState()` 方法中逐一完成此操作。

3. 確保 `activityLayoutInfo` 包含一部分 `DisplayFeature` 屬性，且其中至少有一個屬性為 `FoldingFeature`，否則您不需要執行任何操作：

step1/FoldingStateActor.kt

```
val foldingFeature = activeWindowLayoutInfo?.displayFeatures
    ?.firstOrNull { it is FoldingFeature } as FoldingFeature?
    ?: return
```

4. 計算摺疊位置，確保裝置放置方式確實會影響版面配置，但不超出階層的邊界：

step1/FoldingStateActor.kt

```
val foldPosition = FoldableUtils.getFeaturePositionInViewRect(
    foldingFeature,
    cameraViewfinder.parent as View
) ?: return
```

現在，您已確定 `FoldingFeature` 會影響版面配置，因此需要移動內容。

5. 請檢查 `FoldingFeature` 是否為 `HALF_OPEN`。如果不是，您只需還原內容的位置即可；如果是 `HALF_OPEN`，則需執行另一項檢查，並根據摺疊方向採取不同行動：

step1/FoldingStateActor.kt

```
if (foldingFeature.state == FoldingFeature.State.HALF_OPENED) {
    when (foldingFeature.orientation) {
        FoldingFeature.Orientation.VERTICAL -> {
            cameraViewfinder.moveToRightOf(foldPosition)
        }
        FoldingFeature.Orientation.HORIZONTAL -> {
            cameraViewfinder.moveToTopOf(foldPosition)
        }
    }
} else {
    cameraViewfinder.restore()
}
```

如果摺疊方向為 `VERTICAL`，請將內容移至右邊；如果不是，則請移至摺疊位置上方。

6. 建構並執行應用程式，然後展開裝置並開啟免手持模式，看看內容如何因應方向來移動！
-

步驟 7 · 約 0 分鐘

7. 恭喜！

在本程式碼研究室中，您學到了摺疊式裝置特有的部分功能，例如後置顯示模式或桌面模式，以及如何透過 Jetpack WindowManager 取得這些功能。

您現在可以在相機應用程式上提供優質的使用者體驗了。

其他資訊

- [大螢幕使用入門](#)
- [進一步瞭解摺疊式裝置](#)

參考資料

- [相機預覽](#)
-